VulGuard

Vulnerability Detection and Resolution Tool

1. Description of the Work

VulGuard is an original software extension developed for Microsoft Visual Studio Code (VS Code) that enables automated, AI-assisted vulnerability analysis of C and C++ source code. The work encompasses the complete software system, including its source code, algorithms, analysis pipeline architecture, user interface elements, and associated documentation.

This extension is an original software system implemented in Python. It is designed to detect security vulnerabilities in source code by simultaneously analysing four complementary program representations. It takes as input the raw source code functions alongside three structured program-analysis views, namely Abstract Syntax Trees (AST), Program Dependence Graphs (PDG), and Control Flow Graphs (CFG). The system processes each view through a dedicated transformer encoder. The resulting contextual embeddings are fused by a novel multi-stage fusion mechanism into a single vulnerability prediction.

The system operates through a two-phase pipeline:

- Input capture and preprocessing reads the active editor buffer, normalises line endings, and computes a SHA-256 content hash for snapshot integrity.
- Model inference: dispatches code to the inference server, which runs the Sample-Adaptive View Aware Fusion backbone to produce per-line vulnerability scores.
- Evidence enrichment: constructs a bounded contextual window around each flagged line, scoring evidence snippets by proximity and semantic relevance.

2. Original Elements Claimed for Protection

The authors assert copyright over the following original creative and technical elements:

2.1 Software Architecture and System Design

- The original two-component architecture separating a VS Code extension (TypeScript) from a persistent Python inference server, communicating via a lightweight JSON protocol with SHA-256 revision tokens.
- The separation-of-concerns design principle ensuring that heavy ML inference never blocks the IDE user interface.
- Incremental phase execution with cached artefacts for resumable analysis across edit sessions.

2.2 Technical Innovations

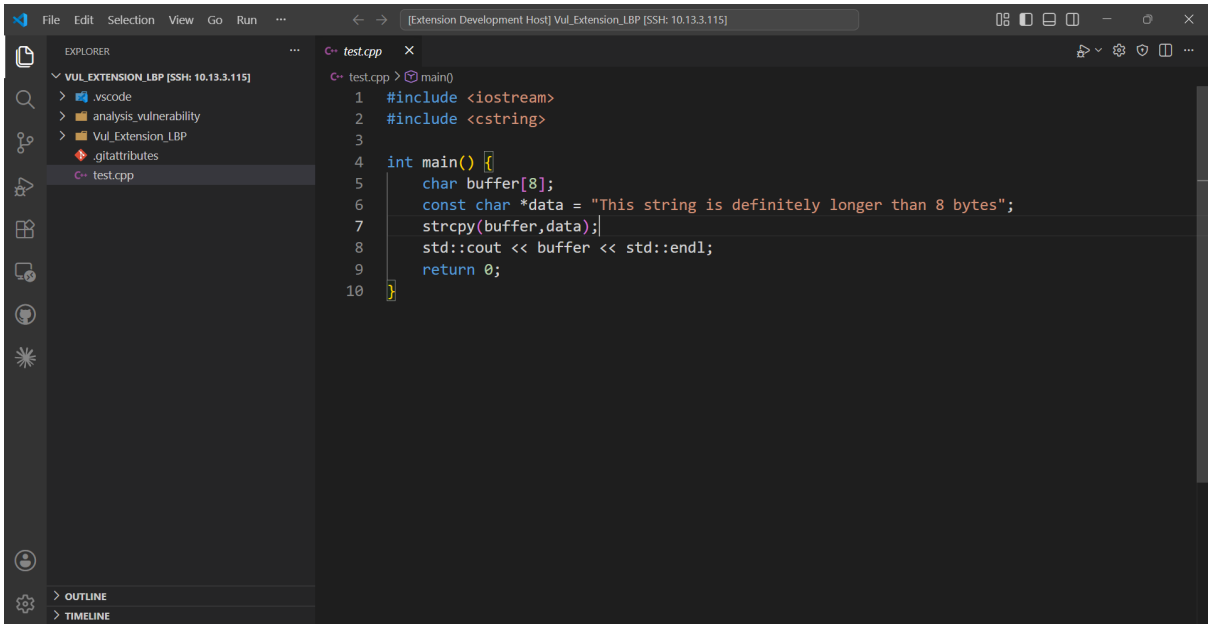
- An originally learned gating mechanism implemented as a four-layer multi-layer perceptron (MLP) that accepts the concatenation of all four view embeddings (dimensionality 4 × hidden_size) and outputs a four-dimensional softmax weight vector.
- The final fused representation is produced by a hand-crafted weighted combination: 60 % of the gated expert output, 30 % of the cross-modal attention mean, and 10 % of the raw view mean.
- A novel novel multi-tokenizer dataset is introduced that simultaneously tokenizes, truncates, and pads four distinct views of each code sample using four independent tokenizer instances.

2.3 Source Code

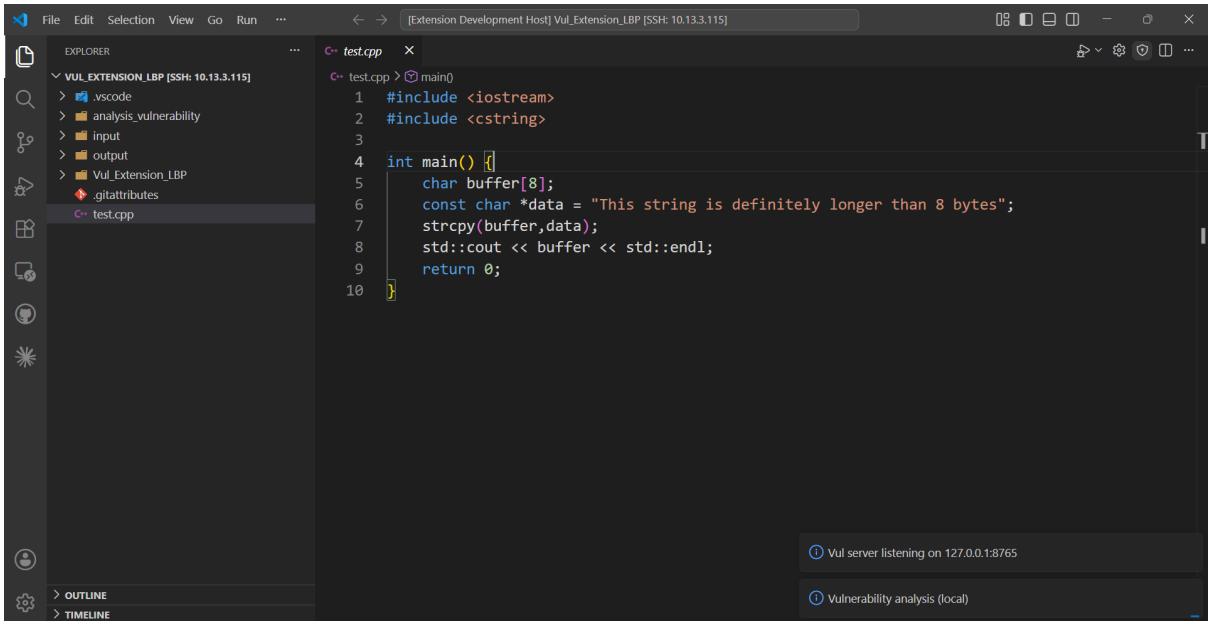
- All original TypeScript source code constituting the VS Code extension, including the inline highlight renderer, patch presentation UI, and server communication layer.
- All original Python source code constituting the inference server, including the preprocessing pipeline, and evidence enrichment logic.

3. Steps for using VulGuard

Step 1: Open your C/C++ File



Step 2: Click on the Shield Icon to start the analysis



Step 3: View the Vulnerability percentage and detected CWE in Output Tab